

Objektinis_programavimas_2

Generated by Doxygen 1.13.2

1 README	1
2 Hierarchical Index	7
2.1 Class Hierarchy	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Class Documentation	13
5.1 Studentas Class Reference	13
5.1.1 Member Function Documentation	14
5.1.1.1 printInfo()	14
5.2 Timer Class Reference	14
5.3 Zmogus Class Reference	15
6 File Documentation	17
6.1 strategijos_bibl.h	17
6.2 humanClass.h	20
6.3 studentClass.h	20
6.4 timer.h	22
Index	23

Chapter 1

README

Programos aprašymas

Programa yra skirta apdoroti studentus ir jų akademinių pasiekimų duomenis, tęsinyje iš objektinis_↔ programavimas_2 realizuotas paveldimumas bei Rule of Five, sugeneruota projekto dokumentacija, realizuoti testai naudojantis Catch2.

Pradinės programos funkcijos:

1. Įvesti visus studentų duomenis ranka
2. Įvesti studentų vardus, pavardes, atsitiktine tvarka generuoti pažymius
3. Atsitiktinai generuoti studentų vardus, pavardes ir pažymius
4. Nuskaityti duomenis iš failo
5. Atsitiktinai generuoti failą su studentų duomenimis

Programoje taip pat pateiktos **trys strategijos**, skirtos testuoti skirtingų programos veikimo laiko priklausomybę nuo naudojamų konteinerių (*vector*, *list*, *deque*) skirstant studentus į pažangius ir nepažangius.

Strategijų aprašymai:

1. Bendro studentų konteinerio (*vector*, *list* ir *deque* tipų) skaidymas (rūšiavimas) į du naujus **to paties tipo** konteinerius: pažangius ir nepažangius.
2. Bendro studentų konteinerio (*vector*, *list* ir *deque*) skaidymas (rūšiavimas) panaudojant tik nepažangių mokinių konteinerį. Tokiu būdu, jei studentas yra nepažangus, jis įkeliamas į nepažangiųjų konteinerį ir ištrinamas iš bendro studentų konteinerio. Atminties atveju ši strategija - efektyvesnė, tačiau dažni trynimai gali būti neefektyvūs tam tikro tipo konteineriams.
3. Bendro studentų konteinerio (*vector*, *list* ir *deque*) skaidymas (rūšiavimas) optimizuojant antrąją strategiją naudojant Standard Template Library (STL)

Naudojimosi instrukcija

1. Susiinstaliuoti `MAKE`
2. Atsidaryti terminalą
3. Klonuoti programos repozitoriją
`git clone https://github.com/augustetum/objektinis_programavimas_1.git`
4. Terminale rašyti žemiau esančią komandą, norint paleisti konteinerių tyrimą

```
make runT
```

Terminale rašyti žemiau esančią komandą, norint paleisti 1 strategiją

```
make run1
```

Terminale rašyti žemiau esančią komandą, norint paleisti 2 strategiją

```
make run2
```

Terminale rašyti žemiau esančią komandą, norint paleisti 3 strategiją

```
make run3
```

Terminale rašyti žemiau esančią komandą, norint paleisti pradinę programą

```
make runP
```

Terminale rašyti žemiau esančią komandą, norint paleisti testavimą (v2.0)

```
make test
```

v2.0

v2.0 sukurta dokumentacija ir atlikti unit testai naudojantis Catch2 testavimo framework. Testai gali būti matomi faile "catchTests.cpp". Pagal žemiau pateiktą kodo iškarpą matoma, jog visi Rule of Five metodai, realizuoti Studento klasėje, veikia korektiškai.

```
Randomness seeded to: 3603547871
=====
All tests passed (7 assertions in 1 test case)
```

v1.5

v1.5 šakoje pridėta nauja klasė - Žmogus. Studento klasė dabar paveldi Žmogaus klasę, todėl atitinkamai buvo pakeisti ir konstruktoriai / destruktoriai. Nauja Žmogaus bei Studento klasė ištestuota su v1.2 versijoje sukurtu testavimo metodu. Rezultatai parodė, jog nauja klasė veikia su anksčiau implementuotais metodais, t.y. testo rezultatai sutampa su žemiau 1.2 versijoje aprašytais to pačio testo rezultatais.

Žemiau pridedamas testo kodas, leidžiantis geriau suprasti testo rezultatus:

```
cout << "Pasirinkote testuoti klasę: " << endl;
cout << "---- Default konstruktorius ----" << endl;
Studentas s1;
cout << "Egzaminas: " << s1.egzaminas() << endl;
cout << endl;
cout << "---- Vardo, pavardės konstruktorius ----" << endl;
Studentas s2("Jonas", "Jonaitis");
cout << "Vardas: " << s2.vardas() << ", pavardė: " << s2.pavarde() << endl;
cout << endl;
cout << "---- Pilnas konstruktorius ----" << endl;
vector<int> pazymiai = {8, 9, 10, 7, 8};
Studentas s3("Petras", "Petraitis", pazymiai, 9);
cout << "Vardas: " << s3.vardas()
<< ", pavardė: " << s3.pavarde()
<< ", egzaminas: " << s3.egzaminas()
<< ", galutinis vid: " << s3.galutinisVid()
```

```

« ", galutinis med: " « s3.galutinisMed() « endl;
cout « endl;
cout « "---- Copy konstruktorius ----" « endl;
Studentas s4(s3);
cout « "s3 kopija - vardas: " « s4.vardas()
« ", pavardė: " « s4.pavarde()
« ", egzaminas: " « s4 egzaminas()
« ", galutinis vid: " « s4.galutinisVid() « endl;
cout « endl;
s4.setVardas("Antanas");
cout « "Po kopijos pakeitimo - s3.vardas: " « s3.vardas()
« ", s4.vardas: " « s4.vardas() « endl;
cout « endl;
cout « "---- Move konstruktorius ----" « endl;
Studentas s5(std::move(s4));
cout « "Movint'as iš s4 - vardas: " « s5.vardas()
« ", pavardė: " « s5.pavarde()
« ", egzaminas: " « s5 egzaminas() « endl;
cout « endl;
cout « "---- Copy assignment operatorius ----" « endl;
Studentas s6;
s6 = s3;
cout « "s6 po s3 kopijavimo - vardas: " « s6.vardas()
« ", pavardė: " « s6.pavarde() « endl;
cout « endl;
cout « "---- Move assignment operatorius ----" « endl;
Studentas s7;
s7 = std::move(s6);
cout « "s7 po movinimo iš s6 - vardas: " « s7.vardas()
« ", pavardė: " « s7.pavarde() « endl;
cout « endl;
cout « "---- Output'o operatorius ----" « endl;
cout « "s3 naudojant operatorių «: " « s3 « endl;
cout « endl;
cout « "---- Input'o operatorius ----" « endl;
string testInput = "Tomas Tomauskas 5 8 9 10 7 9";
istringstream iss(testInput);
Studentas s8;
iss » s8;
cout « "s8 po input: " « s8 « endl;
cout « endl;
cout « "---- Destruktorius ----" « endl;
testuotiDestruktoriu();
cout « "---- Žmogaus konstruktoriaus ----" « endl;
Zmogus z("Pranas", "Pranaitis");
cout « z.vardas() « " " « z.pavarde() « endl;

```

Kaip matoma iš žemiau pridėtos ekrano nuotraukos, testo rezultatai nuo v1.2 nepasikeitė:

Tačiau, neleidžiama sukurti žmogaus objekto, dėl jo abstraktumo:

```

augustetumaite@Augustes-Air objektinis_programavimas_2 % make runP
g++ -std=c++17 -I/opt/homebrew/Cellar/catch2/3.8.1/include -c -o programaFunkcijos.o
    programaFunkcijos.cpp
g++ -std=c++17 -I/opt/homebrew/Cellar/catch2/3.8.1/include -c programa.cpp -o programa.o
programa.cpp:321:28: error: variable type 'Zmogus' is an abstract class
    Zmogus z("Pranas", "Pranaitis");
    ^
./humanClass.h:23:22: note: unimplemented pure virtual method 'printInfo' in 'Zmogus'
    virtual void printInfo() const = 0;
    ^
1 error generated.
make: *** [programa.o] Error 1

```

v1.2

Ankstesnėje v1.2 šakoje implementuoti ir testuoti "Rule of five" metodai bei perdengti įvesties ir išvesties operatoriai. Jų veikimas pritaikytas anksčiau naudotose programos dalyse, tokiose kaip: nuskaitymas iš failo, studentų įvedimas ranka, studentų rezultatų išvedimas.

Žemiau pateikiamos kodo iškarpos, rodančios "Rule of five" bei operatorių perdengimą.

```

//copy konstruktorius
Studentas(const Studentas &s)
: vardas_(s.vardas_),
  pavarde_(s.pavarde_),
  egzaminas_(s egzaminas_),
  galutinisMed_(s.galutinisMed_),
  galutinisVid_(s.galutinisVid_),

```

```

        pazymiuVidurkis_{s.pazymiuVidurkis_},
        pazymiai_{s.pazymiai_}{}

//move konstruktorius
Studentas(Studentas&& s)
: vardas_{std::move(s.vardas_)},
  pavarde_{std::move(s.pavarde_)},
  egzaminas_{s.egzaminas_},
  galutinisMed_{s.galutinisMed_},
  galutinisVid_{s.galutinisVid_},
  pazymiuVidurkis_{s.pazymiuVidurkis_},
  pazymiai_{std::move(s.pazymiai_)}{}

//copy assignment operatorius
Studentas& operator=(const Studentas& s){
    if (this == &s) return *this;

    vardas_ = s.vardas_;
    pavarde_ = s.pavarde_;
    egzaminas_ = s.egzaminas_;
    galutinisMed_ = s.galutinisMed_;
    galutinisVid_ = s.galutinisVid_;
    pazymiuVidurkis_ = s.pazymiuVidurkis_;
    pazymiai_ = s.pazymiai_;

    return *this;
}

//move assignment operatorius
Studentas& operator=(Studentas&& s){
    if (this == &s) return *this;

    vardas_ = std::move(s.vardas_);
    pavarde_ = std::move(s.pavarde_);
    egzaminas_ = s.egzaminas_;
    galutinisMed_ = s.galutinisMed_;
    galutinisVid_ = s.galutinisVid_;
    pazymiuVidurkis_ = s.pazymiuVidurkis_;
    pazymiai_ = std::move(s.pazymiai_);

    return *this;
}

//destruktorius
~Studentas(){
    vardas_.clear();
    pavarde_.clear();
    pazymiai_.clear();
};

//perdengtas įvesties operatorius (panaikintas vardo ir pavardės tikrinimas dėl failuose esančio
//formatavimo, kur vardas ir pavardė GALI turėti skaičius)
std::istream& operator>>(std::istream& cin, Studentas& s){
    string vardas, pavarde;
    vector<int> pazymiai;
    int egzaminas, pazymys;

    cin >> vardas >> pavarde;

    if (cin.eof()) {
        throw std::runtime_error("Netinkamas failo formatas: faile nėra pažymių");
    }

    while (cin >> pazymys){
        if (cin.fail()){
            if (cin.eof()){
                break;
            }
            cin.clear();
            cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            throw std::runtime_error("Netinkamas failo formatas: pažymiai nėra skaitinės reikšmės");
        }
        if (pazymys < 1 || pazymys > 10){
            throw std::runtime_error("Netinkamas failo formatas: pažymiai nėra sveiki skaičiai
            ribose nuo 1 iki 10");
        }
        pazymiai.push_back(pazymys);
    }
    if (!pazymiai.empty()) {
        egzaminas = pazymiai.back();
        pazymiai.pop_back();
    } else {
        throw std::runtime_error("Netinkamas failo formatas: faile nėra pažymių");
    }
    s = Studentas(vardas, pavarde, pazymiai, egzaminas);
    return cin;
}

```



```

}

//perdengtas išvesties operatorius
std::ostream& operator<(std::ostream& out, const Studentas &s) {
    out << std::left << std::setw(20) << s.pavarde() << std::setw(20) << s.vardas() << std::setw(20) <<
    std::fixed << std::setprecision(2) << s.galutinisVid() << std::setw(20) << std::fixed <<
    std::setprecision(2) << s.galutinisMed() << endl;
    return out;
}

```

v1.2 versijoje sukurta klasės testavimo programa leidžia testuoti visas minėtas funkcijas, gaunami rezultatai:

```

---- Default konstruktorius ----
Egzaminas: 0

---- Vardo, pavardės konstruktorius ----
Vardas: Jonas, pavardė: Jonaitis

---- Pilnas konstruktorius ----
Vardas: Petras, pavardė: Petraitis, egzaminas: 9, galutinis vid: 8.76, galutinis med: 8.6

---- Copy konstruktorius ----
s3 kopija - vardas: Petras, pavardė: Petraitis, egzaminas: 9, galutinis vid: 8.76

Po kopijos pakeitimo - s3.vardas: Petras, s4.vardas: Antanas

---- Move konstruktorius ----
Movint'as iš s4 - vardas: Antanas, pavardė: Petraitis, egzaminas: 9

---- Copy assignment operatorius ----
s6 po s3 kopijavimo - vardas: Petras, pavardė: Petraitis

---- Move assignment operatorius ----
s7 po movinimo iš s6 - vardas: Petras, pavardė: Petraitis

---- Output'o operatorius ----
s3 naudojant operatorių <: Petraitis          Petras          8.76          8.60

---- Input'o operatorius ----
s8 po input: Tomauskas          Tomas          8.52          8.60

---- Destruktorius ----
Destruktorius suveikė

```

v1.1

Ankstesnė v1.1 šaka skirta palyginti "class" ir "struct" naudojimą talpinant studento duomenis. Tyrimas ir jo rezultatai pateikiami žemiau, v1.1 implementuota "class" lyginama su v1.0 versijoje naudotu "struct".

Tyrimas vykdytas su 3 strategija (žr. v1.0) bei vektoriaus kontaineriu, vykdytos 3 iteracijos.

"Class" ir "struct" | Tyrimo rezultatai

Programos veikimo laikas su CLASS	Programos veikimo laikas su STRUCT	Studentų kiekis	Optimizavimo vėliava	.exe failo dydis su CLASS (KB)	.exe failo dydis su STRUCT
0.434467s	0.418547s	100 000	-O3	332	329
0.453639s	0.428051s	100 000	-O2	319	315
0.453065s	0.427737s	100 000	-O1	317	331
1.54309s	1.32805s	100 000	-	861	841
1.95142s	1.70978s	1 000 000	-O3	332	329
1.9155s	1.73843s	1 000 000	-O2	319	315
1.93125s	1.7828s	1 000 000	-O1	317	331
7.94422s	6.41446s	1 000 000	-	861	841

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Timer	14
Zmogus	15
Studentas	13

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	13
Timer	14
Zmogus	15

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

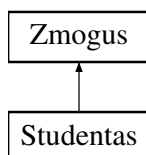
humanClass.h	20
studentClass.h	20
timer.h	22
1_strategija/strategijos_bibl.h	17

Chapter 5

Class Documentation

5.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (string var, string pav, vector< int > paz, int egz)
- **Studentas** (string var, string pav)
- **Studentas** (std::istream &is)
- **Studentas** (const [Studentas](#) &s)
- **Studentas** ([Studentas](#) &&s)
- double **galutinisMed** () const
- double **galutinisVid** () const
- const vector< int > & **pazymiai** () const
- int **egzaminas** () const
- void **setGalutinisV** (double galutVid)
- void **setGalutinisM** (double galutMed)
- void **setEgzaminas** (int egz)
- void **setPazymiai** (vector< int > paz)
- void **skaiciuotiGalutiniSuVid** ()
- void **skaiciuotiGalutiniSuMed** ()
- [Studentas](#) & **operator=** (const [Studentas](#) &s)
- [Studentas](#) & **operator=** ([Studentas](#) &&s)
- bool **operator==** (const [Studentas](#) &s) const
- bool **operator!=** (const [Studentas](#) &s) const
- void **printInfo** () const override
- void **clearStud** ()
- bool **isClear** ()

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (string var, string pav)
- std::string **vardas** () const
- std::string **pavarde** () const
- void **setVardas** (string var)
- void **setPavarde** (string pav)

Friends

- std::ostream & **operator**<< (std::ostream &out, const [Studentas](#) &s)
- std::istream & **operator**>> (std::istream &cin, [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string **vardas_**
- string **pavarde_**

5.1.1 Member Function Documentation

5.1.1.1 printInfo()

```
void Studentas::printInfo () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- studentClass.h
- programaFunkcijos.cpp

5.2 Timer Class Reference

Public Member Functions

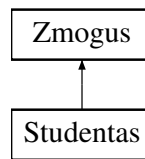
- void **reset** ()
- double **elapsed** () const

The documentation for this class was generated from the following file:

- timer.h

5.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (string var, string pav)
- std::string **vardas** () const
- std::string **pavarde** () const
- void **setVardas** (string var)
- void **setPavarde** (string pav)
- virtual void **printInfo** () const =0

Protected Attributes

- string **vardas_**
- string **pavarde_**

The documentation for this class was generated from the following file:

- humanClass.h

Chapter 6

File Documentation

6.1 strategijos_bibl.h

```
00001 #ifndef STRATEGIJOS1_BIBL_H
00002 #define STRATEGIJOS1_BIBL_H
00003
00004 #include <list>
00005 #include "../studentClass.h"
00006 #include <thread>
00007 #include <functional>
00008
00009 using std::deque;
00010 using std::list;
00011
00012 template <typename Container>
00013 void nuskaitytiFailaT(string fail, Container &studentuSarasas){
00014     string eilut;
00015     int pazymys;
00016     std::stringstream buferis;
00017     ifstream failas(fail);
00018     if (!failas.is_open()){
00019         throw std::runtime_error("Failo nepavyko atidaryti arba jis neegzistuoja šiame
aplankale");
00020     }
00021
00022     buferis « failas.rdbuf();
00023     failas.close();
00024
00025     getline(buferis, eilut);
00026
00027     while(getline(buferis, eilut)){
00028         istringstream eilute(eilut);
00029         string vardas, pavarde;
00030         vector<int> pazymiai;
00031         int egzaminas;
00032
00033         eilute » vardas » pavarde;
00034
00035         if (eilute.eof()) {
00036             throw "Netinkamas failo formatas: faile nėra pažymių";
00037         }
00038
00039         while(true){
00040             eilute » pazymys;
00041             if (eilute.fail()){
00042                 if (eilute.eof()){
00043                     break;
00044                 }
00045                 eilute.clear();
00046                 eilute.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00047                 throw "Netinkamas failo formatas: pažymiai nėra skaitinės reikšmės";
00048             }
00049             if (pazymys < 1 || pazymys > 10){
00050                 throw "Netinkamas failo formatas: pažymiai nėra sveiki skaičiai ribose nuo 1
iki 10";
00051             }
00052             pazymiai.push_back(pazymys);
00053         }
00054         if (!pazymiai.empty()) {
00055             egzaminas = pazymiai.back();
```

```

00056         pazymiai.pop_back();
00057     } else {
00058         throw "Netinkamas failo formatas: faile nėra pažymių";
00059     }
00060     Studentas stud(vardas, pavarde, pazymiai, egzaminas);
00061     studentuSarasas.push_back(stud);
00062     pazymiai.clear();
00063 }
00064 }
00065
00066 template <typename Container>
00067 void rikiuotiPagalVardaT(Container &studentuSarasas){
00068     if constexpr (std::is_same_v<Container, std::list<Studentas>)> {
00069         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00070             return a.vardas() < b.vardas();
00071         });
00072     } else {
00073         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
00074         b) {
00075             return a.vardas() < b.vardas();
00076         });
00077     }
00078 }
00079
00080 template <typename Container>
00081 void rikiuotiPagalPavardeT(Container &studentuSarasas){
00082     if constexpr (std::is_same_v<Container, std::list<Studentas>)> {
00083         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00084             return a.pavarde() < b.pavarde();
00085         });
00086     } else {
00087         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
00088         b) {
00089             return a.pavarde() < b.pavarde();
00090         });
00091     }
00092 }
00093
00094 template <typename Container>
00095 void rikiuotiPagalGalutiniMedT(Container &studentuSarasas){
00096     if constexpr (std::is_same_v<Container, std::list<Studentas>)> {
00097         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00098             return a.galutinisMed() < b.galutinisMed();
00099         });
00100     } else {
00101         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
00102         b) {
00103             return a.galutinisMed() < b.galutinisMed();
00104         });
00105     }
00106 }
00107
00108 template <typename Container>
00109 void rikiuotiPagalGalutiniVidT(Container &studentuSarasas){
00110     if constexpr (std::is_same_v<Container, std::list<Studentas>)> {
00111         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00112             return a.galutinisVid() > b.galutinisVid();
00113         });
00114     } else {
00115         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
00116         b) {
00117             return a.galutinisVid() > b.galutinisVid();
00118         });
00119     }
00120 }
00121
00122 template <typename Container>
00123 void skirstytiStudentusSuTaisPaciaisKonteineriaisT(Container &studentuSarasas){
00124     Container nepazangus;
00125     Container pazangus;
00126     for(Studentas s : studentuSarasas){
00127         if (s.galutinisVid() < 5){
00128             nepazangus.push_back(s);
00129         } else if (s.galutinisVid() >= 5){
00130             pazangus.push_back(s);
00131         }
00132     }
00133
00134     if constexpr (std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
00135     deque<Studentas>)> {
00136         nepazangus.shrink_to_fit();
00137         pazangus.shrink_to_fit();
00138     }

```

```

00138 }
00139
00140 template <typename Container>
00141 void skirstytiStudentusSuVienuKonteineriuT(Container &studentuSarasas, int rikiavimas){
00142     Container nepazangus;
00143     rikiuotiPagalGalutiniVidT(studentuSarasas);
00144     while (studentuSarasas.back().galutinisVid() < 5) {
00145         nepazangus.push_back(studentuSarasas.back());
00146         studentuSarasas.pop_back();
00147     }
00148
00149     if constexpr(std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
deque<Studentas>){
00150         studentuSarasas.shrink_to_fit();
00151         nepazangus.shrink_to_fit();
00152     }
00153
00154
00155     if (rikiavimas == 1) {
00156         rikiuotiPagalVardaT(studentuSarasas);
00157         rikiuotiPagalVardaT(nepazangus);
00158     } else if (rikiavimas == 2) {
00159         rikiuotiPagalPavardeT(studentuSarasas);
00160         rikiuotiPagalPavardeT(nepazangus);
00161     } else if (rikiavimas == 3) {
00162         rikiuotiPagalGalutiniVidT(studentuSarasas);
00163         rikiuotiPagalGalutiniVidT(nepazangus);
00164     } else if (rikiavimas == 4) {
00165         rikiuotiPagalGalutiniMedT(studentuSarasas);
00166         rikiuotiPagalGalutiniMedT(nepazangus);
00167     }
00168
00169 }
00170
00171 template <typename Container>
00172 void skirstytiStudentusT(Container &studentuSarasas){
00173     Container nepazangus;
00174     Container pazangus;
00175     for(Studentas s : studentuSarasas){
00176         if (s.galutinisVid() < 5){
00177             nepazangus.push_back(s);
00178         } else if (s.galutinisVid() >= 5){
00179             pazangus.push_back(s);
00180         }
00181     }
00182
00183     if constexpr(std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
deque<Studentas>){
00184         nepazangus.shrink_to_fit();
00185         pazangus.shrink_to_fit();
00186     }
00187     studentuSarasas.clear();
00188 }
00189
00190 template <typename Container>
00191 void studentuSkirstymas3(Container &studentuSarasas, int rikiavimas){
00192     auto it = std::partition(studentuSarasas.begin(), studentuSarasas.end(), [](const auto &studentas)
{ return studentas.galutinisVid() >= 5; });
00193     Container nepazangus;
00194     nepazangus.assign(it, studentuSarasas.end());
00195     studentuSarasas.resize(std::distance(studentuSarasas.begin(), it));
00196
00197     std::thread t1, t2;
00198
00199     if (rikiavimas == 1) {
00200         std::thread t1([&studentuSarasas]() { rikiuotiPagalVardaT(studentuSarasas); });
00201         std::thread t2([&nepazangus]() { rikiuotiPagalVardaT(nepazangus); });
00202         t1.join();
00203         t2.join();
00204     } else if (rikiavimas == 2) {
00205         std::thread t1([&studentuSarasas]() { rikiuotiPagalPavardeT(studentuSarasas); });
00206         std::thread t2([&nepazangus]() { rikiuotiPagalPavardeT(nepazangus); });
00207         t1.join();
00208         t2.join();
00209     } else if (rikiavimas == 3) {
00210         std::thread t1([&studentuSarasas]() { rikiuotiPagalGalutiniVidT(studentuSarasas); });
00211         std::thread t2([&nepazangus]() { rikiuotiPagalGalutiniVidT(nepazangus); });
00212         t1.join();
00213         t2.join();
00214     } else if (rikiavimas == 4) {
00215         std::thread t1([&studentuSarasas]() { rikiuotiPagalGalutiniMedT(studentuSarasas); });
00216         std::thread t2([&nepazangus]() { rikiuotiPagalGalutiniMedT(nepazangus); });
00217         t1.join();
00218         t2.join();
00219     }
00220
00221     if (t1.joinable()) t1.join();

```

```

00222     if (t2.joinable()) t2.join();
00223 }
00224 #endif
00225

```

6.2 humanClass.h

```

00001 #include <iostream>
00002 using std::string;
00003
00004 class Zmogus {
00005     protected:
00006         string vardas_;
00007         string pavarde_;
00008     public:
00009         Zmogus() {}
00010         Zmogus(string var, string pav) : vardas_{var}, pavarde_{pav} {}
00011         //get'eriai
00012         inline std::string vardas() const { return vardas_; }
00013         inline std::string pavarde() const { return pavarde_; }
00014         //set'eriai
00015         inline void setVardas(string var) { vardas_ = var; }
00016         inline void setPavarde(string pav) { pavarde_ = pav; }
00017
00018         virtual ~Zmogus() {
00019             vardas_.clear();
00020             pavarde_.clear();
00021         }
00022
00023         virtual void printInfo() const = 0;
00024 };

```

6.3 studentClass.h

```

00001 #ifndef MANO_LIB_H
00002 #define MANO_LIB_H
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <vector>
00007 #include <fstream>
00008 #include <string>
00009 #include <numeric>
00010 #include <ctime>
00011 #include <sstream>
00012 #include <chrono>
00013 #include <unordered_set>
00014 #include <limits>
00015 #include <ios>
00016 #include "timer.h"
00017 #include "humanClass.h"
00018
00019 using std::cout;
00020 using std::cin;
00021 using std::string;
00022 using std::vector;
00023 using std::endl;
00024 using std::accumulate;
00025 using std::ifstream;
00026 using std::istringstream;
00027
00028 class Studentas : public Zmogus {
00029     private:
00030         int egzaminas_;
00031         double galutinisMed_;
00032         double galutinisVid_;
00033         double pazymiuVidurkis_;
00034         vector<int> pazymiai_;
00035
00036     public:
00037         Studentas(string var, string pav, vector<int> paz, int egz) :
00038             Zmogus(var, pav, pazymiai_{paz}, egzaminas_{egz}) {
00039             skaiciuotiGalutiniSuMed();
00040             skaiciuotiGalutiniSuVid();
00041         }
00042
00043         Studentas(string var, string pav) : Zmogus(var, pav), egzaminas_(0) {
00044

```



```

00045     Studentas() : Zmogus(), egzaminas_(0) {}
00046     Studentas(std::istream& is);
00047
00048     //copy konstruktorius
00049     Studentas(const Studentas &s)
00050     : Zmogus(s.vardas(), s.pavarde()), egzaminas_{s.egzaminas_},
00051       galutinisMed_{s.galutinisMed_}, galutinisVid_{s.galutinisVid_},
00052       pazymiuVidurkis_{s.pazymiuVidurkis_},
00053       pazymiai_{s.pazymiai_}{}
00054
00055     //move konstruktorius
00056     Studentas(Studentas&& s)
00057     : Zmogus(s.vardas(), s.pavarde()), egzaminas_{s.egzaminas_},
00058       galutinisMed_{s.galutinisMed_}, galutinisVid_{s.galutinisVid_},
00059       pazymiuVidurkis_{s.pazymiuVidurkis_},
00060       pazymiai_{std::move(s.pazymiai_)}{
00061         s.clearStud();
00062     }
00063
00064     //destruktorius
00065     ~Studentas() override{
00066         clearStud();
00067     };
00068
00069     //get'eriai
00070     inline double galutinisMed() const { return galutinisMed_; }
00071     inline double galutinisVid() const { return galutinisVid_; }
00072     inline const vector<int>& pazymiai() const { return pazymiai_; }
00073     inline int egzaminas() const { return egzaminas_; }
00074
00075     //set'eriai
00076     inline void setGalutinisV(double galutVid) { galutinisVid_ = galutVid; }
00077     inline void setGalutinisM(double galutMed) { galutinisMed_ = galutMed; }
00078     inline void setEgzaminas(int egz) { egzaminas_ = egz; }
00079     inline void setPazymiai(vector<int> paz) { pazymiai_ = paz; }
00080
00081     //member funkcijos
00082     void skaiciuotiGalutiniSuVid();
00083     void skaiciuotiGalutiniSuMed();
00084
00085     //copy assignment operatorius
00086     Studentas& operator=(const Studentas& s){
00087         if (this == &s) return *this;
00088
00089         setVardas(s.vardas());
00090         setPavarde(s.pavarde());
00091         egzaminas_ = s.egzaminas_;
00092         galutinisMed_ = s.galutinisMed_;
00093         galutinisVid_ = s.galutinisVid_;
00094         pazymiuVidurkis_ = s.pazymiuVidurkis_;
00095         pazymiai_ = s.pazymiai_;
00096
00097         return *this;
00098     }
00099
00100     //move assignment operatorius
00101     Studentas& operator =(Studentas&& s){
00102         if (this == &s) return *this;
00103
00104         setVardas(s.vardas());
00105         setPavarde(s.pavarde());
00106         egzaminas_ = s.egzaminas_;
00107         galutinisMed_ = s.galutinisMed_;
00108         galutinisVid_ = s.galutinisVid_;
00109         pazymiuVidurkis_ = s.pazymiuVidurkis_;
00110         pazymiai_ = std::move(s.pazymiai_);
00111         s.clearStud();
00112
00113         return *this;
00114     }
00115
00116     //palyginimo operatorius
00117     bool operator==(const Studentas& s) const{
00118         if(vardas_ == s.vardas_ && pavarde_ == s.pavarde_ && egzaminas_ == s.egzaminas_
00119            && pazymiai_ == s.pazymiai_){
00120             return true;
00121         } else {
00122             return false;
00123         }
00124     }
00125
00126     bool operator!=(const Studentas& s) const{
00127         if(vardas_ != s.vardas_ && pavarde_ != s.pavarde_ && egzaminas_ != s.egzaminas_
00128            && pazymiai_ != s.pazymiai_){
00129             return true;
00130         } else {
00131             return false;
00132         }
00133     }

```

```

00130         }
00131     }
00132
00133     //output operatorius
00134     friend std::ostream& operator<<(std::ostream& out, const Studentas &s);
00135
00136     //input operatorius
00137     friend std::istream& operator>>(std::istream& cin, Studentas& s);
00138
00139     void printInfo() const override {
00140         cout << "Vardas: " << vardas() << ", Pavarde: " << pavarde() << endl;
00141     }
00142
00143     void clearStud () {
00144         this->vardas_.clear();
00145         this->pavarde_.clear();
00146         egzaminas_ = 0;
00147         galutinisMed_ = 0;
00148         galutinisVid_ = 0;
00149         pazymiuVidurkis_ = 0;
00150         pazymiai_.clear();
00151     }
00152
00153     bool isClear(){
00154         if(this->vardas_.empty() && this->pavarde_.empty() && pazymiai_.empty() && egzaminas_ ==
00155 0 && galutinisMed_ == 0 && galutinisVid_ == 0 &&
00156         pazymiuVidurkis_ == 0) {
00157             return true;
00158         } else {
00159             return false;
00160         }
00161     };
00162
00163     bool comparePagalPavarde(const Studentas&, const Studentas&);
00164     bool comparePagalEgza(const Studentas&, const Studentas&);
00165
00166     void rodytiRezultatus(vector<Studentas> studentuSarasas);
00167     void generuotiPazymius(vector<Studentas> &studentuSarasas);
00168     void generuotiStudentus(vector<Studentas> &studentuSarasas);
00169     void generuotiFailus(int studentuSkaicius);
00170     string pasirinktiFaila();
00171     void nuskaitytiFaila(string fail, vector<Studentas> &studentuSarasas);
00172     void rodytiVisusRezultatus(vector<Studentas> studentuSarasas);
00173     void testuotiFailuNuskaityma(vector<Studentas> studentuSarasas, int kartai);
00174
00175     //rikiavimo funkcijos
00176     void rikiuotiPagalVarda(vector<Studentas> &studentuSarasas);
00177     void rikiuotiPagalPavarde(vector<Studentas> &studentuSarasas);
00178     void rikiuotiPagalGalutiniMed(vector<Studentas> &studentuSarasas);
00179     void rikiuotiPagalGalutiniVid(vector<Studentas> &studentuSarasas);
00180
00181     void pasirinktiRikiavima(vector<Studentas> studentuSarasas);
00182     void skirstytiStudentus(vector<Studentas> &studentuSarasas);
00183
00184     bool vardoTikrinimas(const string &vard);
00185     void isvestiDuFailus(vector<Studentas> grupel, vector<Studentas> grupe2);
00186     void rikiavimasIrIrasymasVargsiukamsIrKietekams(vector<Studentas> vargsiukai, vector<Studentas>
00187     kietekai);
00188     void testuotiDestruktoriu();
00189 #endif

```

6.4 timer.h

```

00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005
00006 class Timer {
00007     private:
00008         std::chrono::time_point<std::chrono::high_resolution_clock> start;
00009     public:
00010         Timer() : start{std::chrono::high_resolution_clock::now()} {}
00011         void reset() {
00012             start = std::chrono::high_resolution_clock::now();
00013         }
00014         double elapsed() const {return
00015             std::chrono::duration<double>(std::chrono::high_resolution_clock::now() - start).count();
00016         }
00017 };
00018 #endif

```

Index

1_strategija/strategijos_bibl.h, [17](#)

printInfo
 Studentas, [14](#)

README, [1](#)

Studentas, [13](#)
 printInfo, [14](#)

Timer, [14](#)

Zmogus, [15](#)